



iliadev

4 часа назад

Жизнь после RAG: MCP, логи и автоматический анализ тикетов Jira



14 мин



1.5K

Блог компании MTC, Help Desk Software*, Искусственный интеллект, Машинное обучение*, IT-инфраструктура

Кейс

Привет, Хабр! Меня зовут Илья Парамошин, я ведущий инженер в MTC Web Services. В прошлый раз я рассказывал об архитектуре RAG-помощника для технической поддержки: индексации Confluence и Jira, гибридном поиске, генерации ответов с цитатами и оценке качества поиска. А недавно мы опубликовали [продолжение](#) — о том, как построили ИИ-агента для анализа тикетов.

После внедрения RAG инженеры стали быстрее находить инструкции и регламенты, однако расследование инцидентов требовало также работы с логами, базами данных и Jira. Мы не стали переписывать RAG, а начали развивать его как платформу вокруг трех задач: единый интерфейс, доступ к эксплуатационным данным и автоматизация анализа тикетов. Поиск остался ядром, но вокруг появились сервисы для интерактивного и фонового использования. Теперь подробнее разберем архитектуру, причины, по которым вынесли компоненты, и путь тикета от создания до комментария.

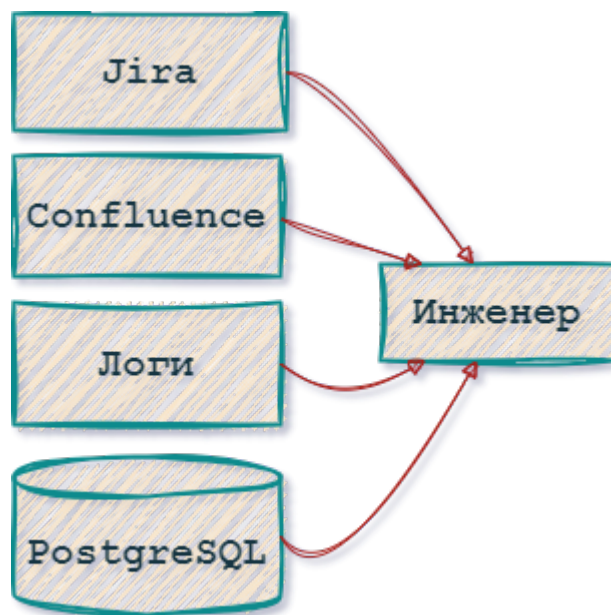


Содержание статьи:

- Когда мы поняли, что только RAG не решит вопрос
- Как устроена архитектура
- MCP как интеграционный слой
- Примеры поведения модели
- Эволюция ядра поиска
- Путь тикета через систему
- Логи, PostgreSQL и Jira — три отдельных контура
- Почему мы отказались от полностью автономного агента
- Удачные решения и неожиданные сложности
- Выводы

Когда мы поняли, что только RAG не решит вопрос

После первых недель эксплуатации мы пришли к выводу: поиск по документации — это лишь первый шаг. Чаще нужно было разобраться в конкретных случаях: почему не дошло SMS, откуда ошибка интеграции, какой статус объекта в БД. Для этого нужны логи, базы данных и состояние сервисов.



Поэтому RAG закрывал только часть процесса. Инженер все равно переходил в Kibana, VictoriaMetrics или PostgreSQL, а затем вручную собирал результаты в комментарий Jira.

Поэтому мы решили развиваться в трех направлениях и создать единый интерфейс для работы со знаниями и инструментами, настроить доступ модели к эксплуатационным данным и автоматизировать обработку тикетов. Именно эти задачи определили архитектуру решения.



Как устроена архитектура

Мы не стали переписывать RAG или строить новую систему с нуля. Вместо этого сохранили поиск по базе знаний как ядро и начали наращивать вокруг него новые возможности: работу с логами, доступ к данным в базах, автоматический анализ Jira и чат через Open WebUI.

RAG стал отдельным сервисом

Первая версия представляла собой Streamlit-приложение со встроенной логикой поиска. Такой подход хорошо работает для прототипа, но плохо масштабируется, когда появляются новые интерфейсы и сценарии.

Поэтому мы вынесли поиск, работу с контекстом и генерацию ответов в отдельный сервис — RAG API. Streamlit, Open WebUI и обработчики Jira стали его клиентами.

Единый интерфейс доступа

Доступ к RAG API реализован через OpenAI-совместимый API. Кроме того, мы настроили собственный MCP-сервер для интеграции с Open WebUI и средой разработки.

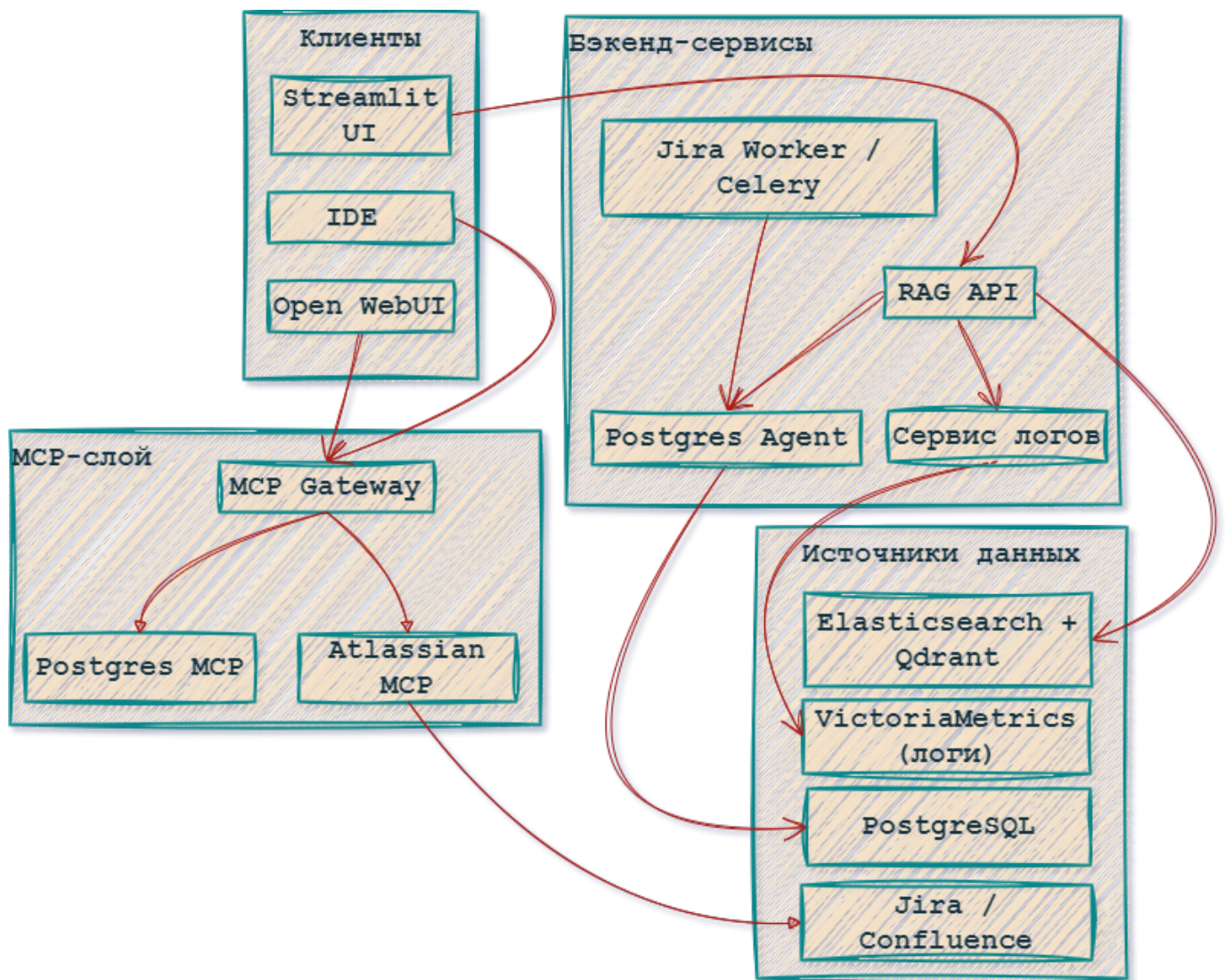
MCP также используем для доступа к PostgreSQL и другим инструментам в чате. При этом к сервису анализа логов и Postgres Retrieval Service Jira-обработчик и RAG-оркестратор обращаются напрямую по HTTP — так проще контролировать таймауты и не плодить лишний уровень в фоновых сценариях. MCP остается слоем для интерактивного чата и IDE.

Выделение сервисов

По ходу развития системы появились отдельные сервисы для анализа логов, обработки тикетов Jira и работы с данными PostgreSQL.

Причина была не только в гибкости архитектуры — поиск занимал разное время в зависимости от сценария. Например, найти информацию по базе знаний обычно занимает несколько секунд, а вот анализ логов или обработка тикета — это нередко несколько запросов к внешним системам и дополнительные вызовы LLM.

Разделение на сервисы помогло изолировать эти операции: длительный анализ логов не влияет на скорость поиска, а обработка тикетов проходит независимо от пользовательских запросов.

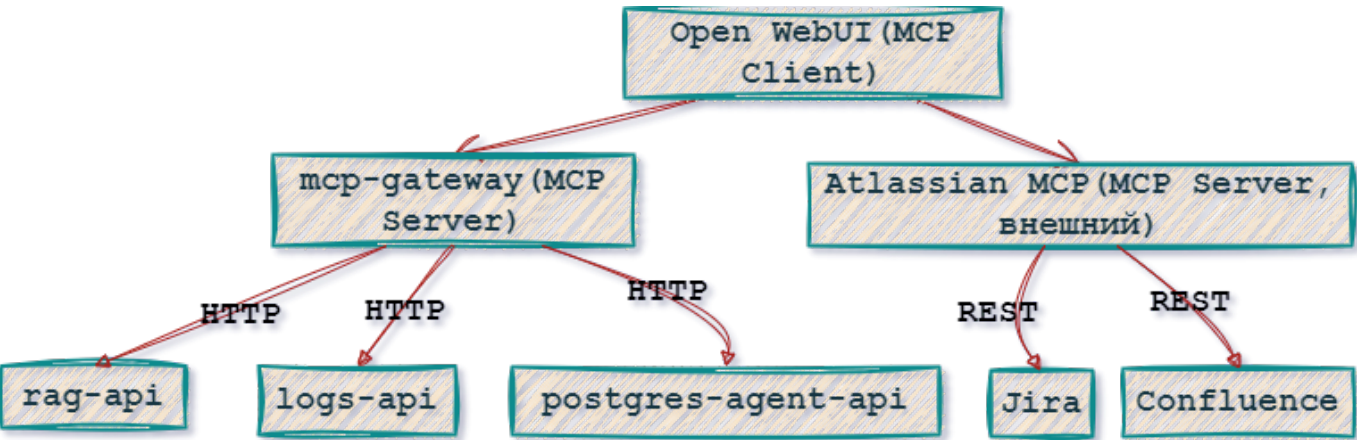


Сервис	Роль
RAG API	Гибридный поиск, переранжирование, генерация ответов с цитатами, оркестрация вызовов логов и Postgres Retrieval
Сервис анализа логов	Отдельное FastAPI-приложение для интерпретации запроса, которая включает: - Каталог — реестр микросервисов с привязкой к источникам логов и полям идентификаторов. - Каскад — из найденных логов вытаскиваем ID и ищем события в связанных сервисах. Ответ для поддержки — структурированную таблицу ошибок и короткий текстовый вывод.
Postgres Retrieval Service	План сценариев → SQL только на чтение через MCP → сборка ответа или Excel

Автоматический анализ Jira	Celery Beat + worker: периодический опрос Jira по JQL, проверка готовности тикета (есть текст и содержательное обращение), маршрутизация — логи, RAG или Postgres — вызов нужных сервисов и публикация результата комментарием в тикет
MCP-сервер	Тонкий адаптер для чата: инструменты → HTTP-запрос к сервисам выше

MCP как интеграционный слой

MCP-протокол упрощает интеграцию с внешними клиентами. Рассмотрим на примере пользовательского интерфейса.



Open WebUI стал основным интерфейсом для инженеров поддержки. По сравнению со Streamlit он дает доступ к истории диалогов, управлению моделями и встроенной поддержке MCP-инструментов. Streamlit остался внутренним стендом для разработки и отладки RAG-поиска — там удобно экспериментировать с параметрами retrieval (порог релевантности, режим BM25/вектор, rerank) и смотреть, какие фрагменты документов попали в контекст ответа.

А еще Open WebUI выступает в качестве универсального MCP-клиента. Пользователь работает в одном окне, а модель при необходимости обращается к поиску по базе знаний, анализу логов, Jira или другим сервисам.

При подключении MCP-сервера Open WebUI получает список доступных инструментов и их описание. Мы передаем эту информацию модели вместе с системным промптом.

Дальше модель сама решает, нужен ли ей внешний инструмент для ответа на запрос пользователя. Если по тексту запроса недостаточно информации для выбора инструмента, модель сначала задает уточняющий вопрос.

На текущий момент мы используем два основных набора инструментов.

1. ИИ-помощник (RAG & Logs):

- `rag_query`: гибридный поиск по базе знаний с цитированием (опционально с обогащением из Postgres);
- `analyze_logs`: HTTP-запрос к сервису анализа логов;
- `analyze_jira_ticket`: автоматический анализ тикетов по логике Celery-обработчика.
- `postgres_agent`: выполнение sql запросов к подключенным по mcp базам данных.

2. **Atlassian MCP**: работаем напрямую с Jira и Confluence без написания JQL-запросов вручную. Поиск тикетов, страниц документации и статусов.

Благодаря этому можно расширять систему без изменений в пользовательском интерфейсе. Если зарегистрировать новый сервис как MCP-инструмент, он становится доступен всем клиентам, поддерживающим протокол MCP.

Но на практике мы столкнулись с тем, что просто подключить инструмент недостаточно. Чтобы модель правильно выбирала инструменты, она должна понимать их назначение. Поэтому вместе с добавлением нового MCP-инструмента мы обычно корректируем системный промпт и добавляем примеры типовых сценариев.

Примеры поведения модели

На основе системного промпта и описания инструментов модель сама выбирает стратегию:

Вопрос: «Почему не доставилось SMS для client_id 12345?»

Действие: модель видит описание `analyze_logs` и вызывает инструмент для проверки логов.

Вопрос: «Как работает механизм повторной доставки?»

Действие: модель выбирает `rag_query` для поиска информации в базе знаний.

Запрос: «Покажи последние открытые тикеты по SMS-шлюзу.»

Действие: модель направляет запрос в Atlassian MCP (Jira).

Эволюция ядра поиска

Несмотря на все нововведения, RAG остается центральным компонентом системы. Через него проходят как запросы инженеров в чате, так и значительная часть автоматических сценариев.

Общая логика работы практически не изменилась:

- мы преобразуем пользовательский запрос в поисковый;
- выполняем поиск по нескольким индексам;
- объединяем и ранжируем результаты;
- передаем найденный контекст в LLM для генерации ответа.



Более заметные изменения произошли на уровне хранилищ поиска.

В первой версии мы использовали PostgreSQL с pgvector для хранения документов и векторного поиска. Это удобно для прототипа: вся система работает на минимальном наборе инфраструктуры. Но по мере роста объема данных требования к лексическому и векторному поиску начали расходиться. Для текстового поиска понадобилась более гибкая настройка BM25 и поисковых запросов, а для векторного — независимое управление индексами и параметрами.

В результате мы разделили эти задачи между специализированными системами.

При этом логика гибридного поиска не изменилась: запрос одновременно отправляется в Elasticsearch и Qdrant, затем мы объединяем результаты и передаем в LLM.

Индексацию мы по-прежнему выполняем в фоне. Коннекторы к Confluence и Jira регулярно загружают только изменившиеся документы, после чего мы индексируем текст в Elasticsearch и сохраняем эмбединги в Qdrant.

Для пользователей эти изменения практически незаметны, но переход на специализированные хранилища позволил нам независимо развивать лексический и векторный поиск и сохранять качество выдачи по мере роста базы знаний.

Путь тикета через систему

А теперь давайте посмотрим, как все компоненты работают вместе на реальном примере. Возьмем типичный тикет поддержки:

Тема: ошибка доставки сообщения

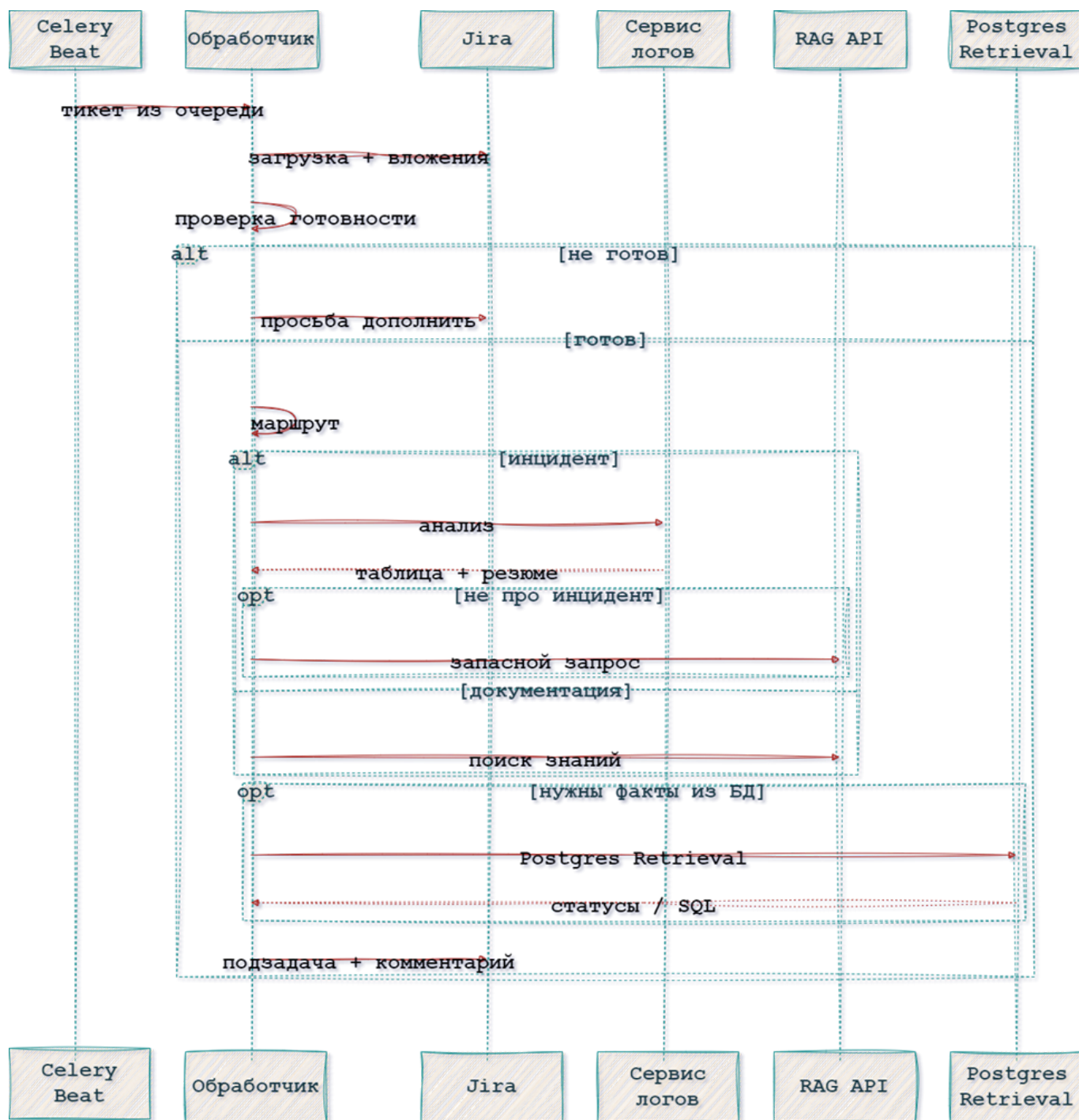
Описание: клиент не получил SMS, client_id=12345, примерно вчера после 14:00.
Повторите проверку.

Как было раньше

Инженер открывал Jira, искал события по client_id в логах, проверял состояние связанных объектов в базе данных, искал похожие случаи в документации и после этого вручную собирал комментарий с результатами. Обычно на это уходило 15–40 минут, иногда дольше.

Как работает сейчас

Ниже — **фоновая обработка** (Celery Beat + worker). Это отдельный контур от интерактивного чата в Open WebUI.



1. Планировщик. Celery Beat опрашивает Jira по JQL из конфига и отбирает новые тикеты и кандидатов на переанализ. Задачи ставятся **в отдельную очередь на Jira-проект** (jira_analysis.project1, jira_analysis.project2 и так далее), чтобы тяжелый поток одного проекта не блокировал остальные.

Чтобы Beat не забивал очередь повторами, пока тикет уже обрабатывается, перед постановкой задачи в Redis создается ключ jira_analysis:enqueue_dedupe:{TICKET}:{reason} (операция SET NX, TTL по умолчанию 2 часа). Пока ключ существует, повторная задача не ставится. Когда воркер завершает работу — ключ удаляется.

2. Проверка готовности. В нашем случае это два последовательных шага:

Шаг А (детерминированный, без LLM): в заголовке или описании есть непустой текст. Опционально в конфиге можно задать `required_statuses` — тогда анализируются только тикеты в указанных статусах. Если текста нет совсем — в тикет один раз уходит комментарий с просьбой дополнить описание (`request_info`); факт отправки храним в Redis с TTL, чтобы не спамить автору. Полный анализ на этом шаге не стартует.

Шаг В (LLM): эндпоинт `suggest-clarification` оценивает, есть ли в тикете содержательное обращение (инцидент, проблема, запрос с контекстом), а не формальная фраза вроде «перезвоните». Это не сам анализ, а легкая предпроверка: «стоит ли запускать разбор?». Если модель считает, что данных мало — снова `request_info`, полный анализ не запускаем.

Сбой шага В (LLM недоступна, ошибка, таймаут) — отдельный сценарий. Здесь работает `fail-open`: предпроверку пропускаем и всё равно запускаем полный анализ — тикет переходит к шагу 3 (маршрутизация: логи и/или RAG, поиск по базе знаний, генерация ответа, комментарий в Jira). Если основной пайплайн тоже не сможет ответить — тикет завершится с ошибкой воркера, а не с просьбой автору «допишите описание», что было бы неправильной реакцией на инфраструктурный сбой.

3. Маршрут: логи или RAG. Понятно, что не каждый тикет одинаковый. В одном нужно посмотреть, что произошло в проде прямо сейчас, в другом — найти ответ в документации. Поэтому для части потоков поддержки мы включили маршрутизацию: система сначала решает, куда идти — в логи или в базу знаний. Для остальных потоков всегда выполняется полный RAG: поиск по Confluence или Jira и генерация ответа.

Режим можно задать в конфиге отдельно на каждый поток:

- только база знаний — тикет всегда идет в RAG, логи не трогаем;
- только логи — сначала разбор событий в эксплуатационных системах, без векторного поиска;
- авто — LLM по тексту тикета выбирает между двумя путями. «Не дошло SMS по `client_id`» обычно уходит в логи, «как работает лимит на рассылку» — в документацию.

Запросы к PostgreSQL у нас отдельная линия: они могут дополнить ответ из логов или RAG фактами из БД, а могут быть самостоятельной задачей — например, когда в тикете просят выгрузить отчет в Excel.

4. Сервис логов. Сервис анализа разбирает запрос, находит нужный микросервис в каталоге, извлекает данные, при необходимости выполняет каскадный поиск по связанным сервисам, агрегирует и резюмирует информацию. На выходе мы получаем структурированный результат, а не многостраничный лог-файл.

5. Публикация. Результат оформляется в виде подзадачи и комментария с таблицей, резюме и ссылками на источники. Для типовых сценариев система просто обновляет существующий комментарий.

6. Запасной путь. Маршрутизатор не идеален: тикет иногда уходит в логи, хотя по смыслу это вопрос по документации. Чтобы не оставлять пустой комментарий, сервис логов делает вторую проверку — похоже ли обращение на разбор инцидента в проде (например, «не дошло сообщение»). Если нет, он сообщает обработчику, что логи здесь бесполезны, и тот автоматически переключается на полный RAG — поиск по базе знаний и ответ с цитатами.

Схема работы через OpenWebUI

Инженер отправляет запрос в Open WebUI: «почему не дошло SMS client_id 12345 вчера». Модель через MCP вызывает analyze_logs или rag_query. Тот же вопрос в чате от коллеги даст тот же результат.

Пример запроса от клиента:

*«Требуется отправить сообщение на 100+ номеров. Функция https://***.mts.ru/http-api/v1/messages при попытке передать более 10 номеров выдает ошибку 400.*

Можно ли передать за 1 раз крупное кол-во номеров? Если невозможно, то каким образом рекомендуется делать рассылку для 100+ человек, при условии что надо проверять статус отправки и доставки СМС сообщений?»

Комментарий в Jira:

Анализ ИИ-помощника СТП

Краткий вывод

Ошибка 400 возникает штатно: endpoint <https://mts.ru/http-api/v1/messages> (HTTP_SIMPLE) технически ограничен отправкой не более 10 сообщений в одном запросе (регламент, Confluence Документация REST API). Для массовой рассылки (100+ номеров) необходимо использовать буферный endpoint <https://mts.ru/http-api/v1/b/messages/> или пакетный режим через HTTP_BATCH (загрузка CSV) (регламент, JIRA [-36236](#)).

Факты из БД (Postgres)

1. client_id = 1101626, msisdn = 7985, inn = 770, ls = 27778 ()
2. Активное подключение HTTP_SIMPLE (логин gw...), используемое клиентом для текущего запроса ()
3. Активное подключение HTTP_BATCH (логин me... — пример из контекста, у клиента аналогично настроен для пакетов) ()
4. Активные пакеты SMS в таблице pack со статусом ACTIVE, лимиты до 400 000+ сообщений ()
5. Счетчик отправленных сообщений в counter обновляется корректно, лимиты не исчерпаны ()

Суть регламента / ключевые факты

1. Endpoint POST /http-api/v1/messages (HTTP_SIMPLE) поддерживает отправку **не более 10 SMS** в одном запросе. Превышение лимита возвращает ошибку 400 (Bad Request) (Confluence Документация REST API, JIRA [-36236](#)).
2. Для отправки более 10 SMS в одном запросе необходимо использовать endpoint буфера: POST /http-api/v1/b/messages/ (Confluence Документация REST API, JIRA [-36236](#)).
3. Альтернативный вариант — пакетная отправка через HTTP_BATCH (загрузка CSV-файла) для массовых рассылок (JIRA [-36236](#)).
4. Для отслеживания статусов отправки и доставки необходимо использовать метод GET/POST /http-api/v1/messages/info или слушать коллбэки (Confluence Документация REST API).

Факты из логов

1. portal (2026-05-20T12:45:40.994758+00:00): Запрос от клиента 1101626 (MSISDN 7985) на endpoint /http-api/v1/messages с описанием ошибки 400 при >10 номерах (логи).
4. counter (2026-05-20T12:48:00.296223+00:00): Создание счетчика Counter для клиента 1101626, пакет 477490 (100 000 сообщений), статус ACTIVE (логи).

Ссылка для просмотра логов

Рекомендуемые действия

1. Изменить URL запроса в интеграции клиента с <https://mts.ru/http-api/v1/messages> на буферный <https://mts.ru/http-api/v1/b/messages/> для отправки более 10 номеров в одном запросе.
2. При использовании пакетной отправки (HTTP_BATCH) сформировать CSV-файл с колонками msisdn, text, client_id и загрузить его через соответствующий API endpoint.
3. Для проверки статусов отправки и доставки использовать метод POST /http-api/v1/messages/info с передачей msg_ids или int_ids (внешних или внутренних идентификаторов сообщений).
4. Проверить наличие у клиента активного подключения HTTP_BATCH и актуальность пакетов SMS в системе (таблицы connection, pack), при необходимости — оформить новый пакет через CRM или UI.
5. Если клиент использует HTTP_SIMPLE для массовых рассылок, объяснить, что это штатное ограничение, а не техническая ошибка, и рекомендовать переход на буферный endpoint.

Ссылки на справку

- Как настроить интеграцию по REST API

Дополнительные материалы: подзадача [-69656](#) (подробные выдержки и расширенный контекст).

► Пример анализа через Open WebUI

Логи, PostgreSQL и Jira — три отдельных контура

Сервис анализа логов

Мы оформили анализ логов как отдельное FastAPI-приложение, которое вызывают RAG API, обработчик Jira и MCP.

Изначально логика работала внутри основного приложения, но быстро выяснилось, что длительные запросы к логам блокируют обычные RAG-запросы. Когда мы выделили сервис в отдельный процесс, проблемы с таймаутами и перегрузкой исчезли.

Внутри сервис использует каталог микросервисов с описанием источников логов и поддерживает каскадный поиск: если ошибка обнаружилась в одном сервисе, мы автоматически ищем связанные события в соседних системах.

На выходе мы формируем не набор строк, а структурированное резюме с таблицей ошибок и ссылками на первоисточник. Этот подход оказался значительно надежнее, чем попытки передавать модели большие фрагменты логов напрямую.

Postgres Retrieval Service

Документация отвечает на вопрос, как должна работать система. Но при расследовании инцидентов часто нужны факты из эксплуатационных БД: статус кампании, результаты обработки, текущие ограничения или ошибки валидации.

Для этого мы создали отдельный Postgres Retrieval Service. Он получает задачу на естественном языке, строит сценарий получения данных и выполняет SQL-запросы только на чтение через MCP.

Сервис работает в двух режимах:

- как источник фактов для обогащения RAG-ответов;
- как самостоятельный инструмент для подготовки отчетов и выгрузок. Все запросы ограничены политиками безопасности: только SELECT, лимиты на объем данных и дополнительная проверка SQL перед выполнением.

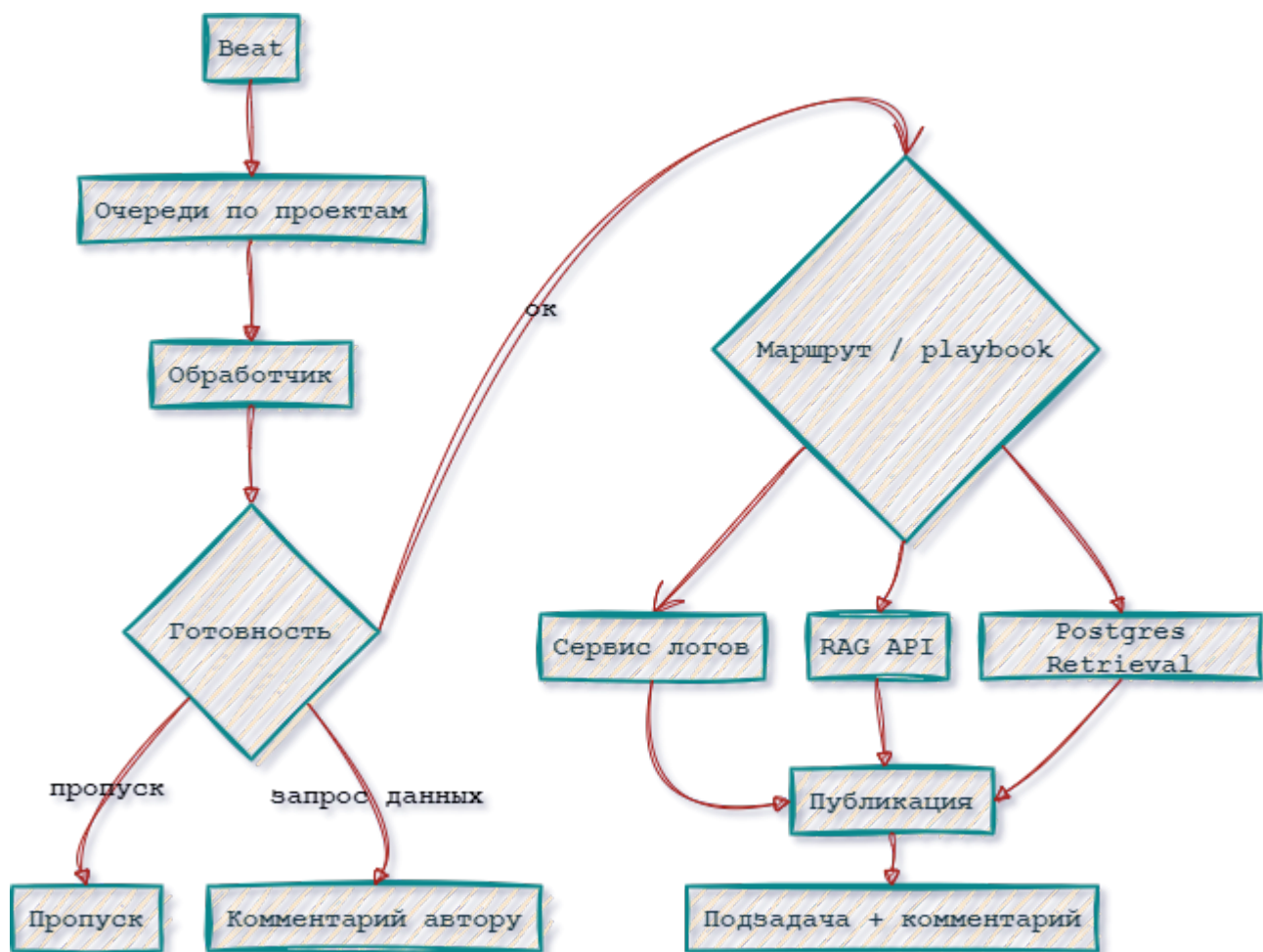
Все запросы ограничены политиками безопасности: только SELECT, лимиты на объем данных и дополнительная проверка SQL перед выполнением.

Автоматический анализ Jira

Автоматический анализ реализован как отдельный контур на базе Celery.

Планировщик регулярно проверяет Jira, помещает новые тикеты в очередь и запускает обработку по сценарию: сначала проверяет корректность тикета, затем выбирает маршрут, выполняет анализ и публикует результат.

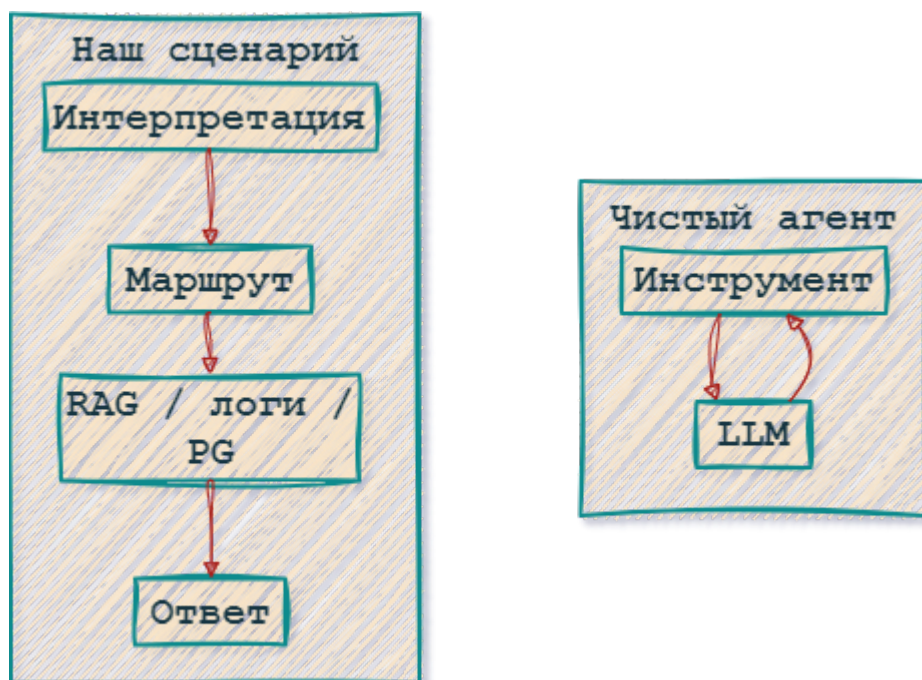
Для устойчивой работы мы используем отдельные очереди по проектам, дедупликацию задач через Redis и обновление существующих комментариев вместо создания новых.



Почему мы отказались от полностью автономного агента

В индустрии при проектировании LLM-систем часто противопоставляют два подхода: автономного агента, который сам выбирает инструменты в цикле, и жесткий сценарий с фиксированной цепочкой шагов.

Мы выбрали промежуточный путь. Последовательность шагов фиксирована: интерпретация запроса, выбор маршрута, обращение к поиску, логам или PostgreSQL, формирование ответа. А внутри каждого шага решение принимает LLM — нужны ли логи, какие SQL-запросы построить, как оформить результат.



Мы не пошли по пути автономного агента для Jira, так как поддержке нужна воспроизводимость — по логам должны быть видны каждый шаг, таймауты и выбранный маршрут. Однажды автономный агент на тикете ушел в цепочку вызовов, которую мы не смогли воспроизвести по трейсу, а это неприемлемо для продакшна. Кроме того, необходим контроль таймаутов: анализ логов и работа с PostgreSQL могут длиться минутами.

Агентный цикл уместен внутри Postgres Retrieval Service при выполнении сложных выгрузок, куда входит построение плана, параллельные SQL-запросы, сборка результата, формирование Excel. В этом случае цикл LLM и MCP оправдан и изолирован в отдельном сервисе.

Жесткий сценарий мы используем для автоматического анализа Jira и типового RAG-запроса. LLM участвует на отдельных шагах, но граф переходов зафиксирован в коде и конфигурации.

Удачные решения и неожиданные сложности

Что сработало:

- **Отдельные сервисы для логов и работы с базами данных.** Разделение позволило нам использовать разные таймауты, независимо разворачивать компоненты и не смешивать быстрые сценарии с длительными.
- **MCP как интеграционный слой.** MCP оказался удобным способом подключать инструменты к Open WebUI и средам разработки без изменений в RAG-ядре. Новый

инструмент обычно сводится к HTTP-эндпоинту и небольшой обертке на стороне MCP.

- **Жесткий сценарий для автоматического анализа Jira.** Для поддержки важнее воспроизводимость, чем максимальная автономность агента. Когда маршрут обработки зафиксирован, проще понимать, что произошло, искать ошибки и контролировать время.
- **Табличное представление результатов анализа логов.** Короткая таблица с кодами ошибок, идентификаторами и ссылками на первоисточник оказалась намного полезнее, чем большие фрагменты логов, которые генерировала модель в первых версиях решения.
- **Запасной путь между логами и RAG.** Классификатор маршрута в автоматическом анализе Jira периодически ошибается: тикет уходит в логи, хотя по смыслу это вопрос по документации. Запасной переход с анализа логов на поиск по базе знаний заметно сократил количество пустых или бесполезных комментариев.

Что оказалось сложнее, чем мы ожидали:

- **Вызов инструментов через MCP не гарантирован.** Даже когда нужный инструмент подключен, модель нередко отвечает текстом в чате, вместо того чтобы вызвать `analyze_logs` или другой инструмент. Системные инструкции и примеры помогают, но полностью убрать такое поведение не получается.
- **Классификация тикетов ошибается.** Маршрут «логи или документация» не всегда определяется корректно. Поэтому запасной путь остался обязательной частью сценария, а не временным решением.
- **Длительные операции никуда не исчезли.** Анализ логов и сложные запросы к базам данных — это по-прежнему медленные операции, они могут длиться минуты. Выделение в отдельные сервисы помогает их изолировать, но не ускоряет.
- **Переранжирование дает качество ценой скорости.** LLM-переранжирование действительно улучшает результаты поиска, но в реальных условиях часто увеличивает задержку в несколько раз. Поэтому в реальном применении мы используем его осторожнее, чем во время экспериментов.
- **Новая архитектура добавляет новые точки поддержки.** Переход на Elasticsearch и Qdrant, появление отдельных сервисов и нескольких MCP-серверов сделали систему гибче, но увеличили объем эксплуатации.

Выводы

Когда мы только начинали, казалось, что главное — построить качественный RAG по документации и истории обращений. На практике поиск по базе знаний оказался только частью решения. Большая часть времени инженера поддержки уходит на работу с эксплуатационными данными. Поэтому мы постепенно пришли к сервису анализа логов, Postgres Retrieval Service, автоматическому анализу Jira и MCP-интеграциям. При этом поиск остался фундаментом всей системы.

Если бы мы сейчас могли вернуться в исходную точку, то дали бы себе несколько советов.

Сразу отделять RAG от пользовательского интерфейса

Streamlit хорошо подходит для прототипа, но быстро появляются новые потребители: чат, IDE, фоновые обработчики. Если поиск живет внутри интерфейса, каждая новая точка входа превращается в отдельный проект. Решение выделить RAG API оказалось одним из самых полезных.

Не пытаться решить все задачи через RAG

Поиск по документации и расследование инцидентов — разные задачи. Если ответ требует логов или данных из PostgreSQL, лучше дать системе прямой доступ к этим источникам, чем индексировать все в векторное хранилище.

Разделять сценарии по времени выполнения

Поиск обычно занимает несколько секунд. Анализ логов и работа с базами уже на порядок больше. Если запускать все в одном процессе, получаются таймауты, очереди и плохой пользовательский опыт. Если сценарии отличаются по времени — разделяйте их физически.

Использовать MCP как интеграционный слой, а не как основу архитектуры

MCP удобен для подключения инструментов к чатам и IDE. Но бизнес-логику оставляйте в своих сервисах. Тогда один и тот же функционал будет доступен через чат, API или фоновые обработчики без переписывания.

Использовать агентность внутри задачи, а не вместо архитектуры

Лучшие результаты мы получили не от полностью автономных агентов, а от комбинации жестких сценариев и LLM на отдельных шагах. Маршрутизация, контроль таймаутов и публикация остаются детерминированными. Агентные циклы хорошо работают внутри отдельных задач — например, при построении сложной SQL-выгрузки.

Быть готовым к доработкам после запуска

Самые эффективные улучшения мы получили не во время разработки, а после анализа ошибок. Мы разбирали неудачные ответы, корректировали маршрутизацию, добавляли новые сценарии и уточняли поведение агентов.

Возможно, именно в этом и заключается главный практический результат подобных систем сегодня. Они не заменяют инженера поддержки, но позволяют постепенно передавать машине все большую часть рутинной работы, оставляя человеку принятие решений и контроль сложных случаев. А вы как считаете?

Теги: искусственный интеллект, служба поддержки, ai-ассистент, llm, mws gpt, ии-помощник, техподдержка, rag

Хабы: Блог компании МТС, Help Desk Software, Искусственный интеллект, Машинное обучение, IT-инфраструктура

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



256K+

Охват за 30 дней

МТС



3

Карма

@iliadev

Пользователь

Подписаться



Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



Shannon

19 часов назад

habrGPT. Обучим LLM 0.5B с нуля на статьях Хабра с помощью nanochat от Карпатого. Обучение fp8 дома и сравнение с bf16



17 мин



15K



+75



120



23



interpres

23 часа назад

Как устроен двигатель внутреннего сгорания (завершение)



Средний



10 мин



11K

Обзор

Перевод



+37



44



1



kliment_01

22 часа назад

Как настроить Raspberry на актуальной версии Raspberry Pi OS



8 мин



12K



+34



26



2



kaftanati

15 часов назад

Умный летний дом своими руками (буквально)



Простой



20 мин



12K

Ретроспектива



+33



38



13



wmlab
13 часов назад

Планета, которой не было



Простой



13 мин



12K



+32



8



11



Li_Lu09
15 часов назад

«Авторский огонёк» Хабра: почему мы зажигаем авторские сердца



Простой



21 мин



8.4K



+29



5



9



TrexSelectel
23 часа назад

Mageia 10: 15 лет спустя форк Mandriva продолжает развиваться. Что нового?



7 мин



11K



+29



2



1



Flampanzer
4 часа назад

Управление ключами SSH — вызовы и эволюция подходов



9 мин



5.9K

Обзор



+24



19



1



ProstoKirReal
4 часа назад

Сложно о простом. Все, что бы вы хотели знать о SFP модулях. Часть 6. Топ-10 частых ошибок при выборе и эксплуатации SFP



Средний



9 мин



7.7K

Обзор



+21



33



4



monobogdan
20 часов назад

Часы за 4.000 рублей с оффлайн-картами, памятью для музыки и подзарядкой раз в месяц — обзор на Cubot GT5



6 мин



16K

Обзор



+19



21



33

Показать еще

ВАКАНСИИ КОМПАНИИ «МТС»

Senior SRE/DevOps Инженер [MWS Cloud Platform]

MTC Web Services · Москва · Можно удаленно

Системный инженер C/C++ в команду Storage Core SPDK [MWS Cloud Platform]

MTC Web Services · Москва · Можно удаленно

Middle/Senior DevOps Engineer [MTC Веб Сервисы]

MTC Web Services · Москва

Больше вакансий на Хабр Карьере



Настройка языка

Техническая поддержка